

## Addendum: Pre-Phase-10- Close Gap Fixes

**Context:** Phases 2–9 are built and tested. Before Phase 10 (hardening/deploy) is called done, fix the gaps below — they're correctness/trust issues in the core flow, not new features. They were found by tracing full round-trips through the spec (what actually happens end-to-end) rather than reviewing each phase in isolation, which is why they weren't caught earlier.

**Process for this addendum:** for each gap, **audit first** — check what's actually implemented before writing new code. Some of these may already be partially or fully handled; report per-gap status (already solved / partially solved / missing) before touching anything. Don't rebuild what already works.

Fix in priority order — 1 and 2 are data-integrity/trust issues, do those first.

---

**Gap 1 (highest priority) —  
Confirm/Edit/Cancel buttons may  
violate Telegram's callback\_data limit**

**Problem:** Telegram's `callback_data` field has a hard 64-byte limit. If the pending entry's actual data (fields, values) is encoded directly into the button payload, it will silently fail ( `BUTTON_DATA_INVALID` ) the moment an entry has any real content — which is every entry.

**Audit first:** check how `callback_data` is currently constructed for the Confirm/Edit/Cancel keyboard. If it already uses a short reference (see fix below), this gap is already closed — confirm and move on.

**Fix if missing:** `callback_data` should only ever carry a short reference key, e.g. `confirm:<draft_id>` , `edit:<draft_id>:<field>` , `cancel:<draft_id>` — where `draft_id` is a primary key into a server-side store (see Gap 2). Full field data is resolved from that store, never from the button itself.

**DoD:** log an entry with a long doubt text / notes field, confirm the buttons still work (no silent Telegram API error in logs).

---

## Gap 2 — No defined state for "pending write" (the thing Edit/Confirm/Cancel act on)

**Problem:** Phase 7 says Confirm writes to Notion and Edit "re-prompts for just the fields to change" — but

nowhere is it specified where that in-progress draft actually lives between the preview message and the button tap.

**Fix:** add a `pending_writes` table (SQLite, same DB as the mirror or a separate small one):

```
pending_writes(draft_id TEXT PK, chat_id
INTEGER, action TEXT, database TEXT,
               fields_json TEXT,
filters_json TEXT, preview_message_id
INTEGER,
               created_at TEXT, expires_at
TEXT)
```

- Created when a preview is shown, referenced by `draft_id` in the button callback\_data (Gap 1).
- **Add a TTL** (e.g. 15 min) — if a draft is abandoned (user never taps a button), auto-expire it and edit the original message to say "expired, send again" rather than leaving a stale draft the bot might act on later.

**DoD:** start logging an entry, don't tap any button, wait past the TTL, confirm the bot correctly expires it instead of leaving it live forever.

---

## Gap 3 — Edit flow has no formal way to know *which* field changed

**Problem:** The Phase 5 intent JSON contract only has

`action: log_execution | log_doubt |`

`log_revision | query | set_context | unknown` —

there's no `edit_field`, so "re-prompt for just the fields the user wants to change" has no defined mechanism.

**Fix — do this the reliable way, not the LLM-parse way:**

when Edit is tapped, show an inline keyboard listing the draft's current field names (pulled from

`pending_writes.fields_json`). Tapping a field name

prompts "send the new value for `<field>`." The next plain-text message from that chat is applied **directly** to that one field in the draft (no LLM parse needed for simple fields — numbers/text just overwrite). For relation fields (Chapter, etc.), re-run the same fuzzy-match resolution used in the original flow (Phase 7 step 5). Re-render the preview after each edit; user can edit multiple fields before confirming.

This avoids adding a whole new "freeform edit" intent type and its own parse-failure surface — deterministic button + direct value assignment is more reliable for a personal tool than making the LLM guess what "actually it was 18 correct" refers to.

**DoD:** log an entry, tap Edit, change one numeric field via the field-picker, confirm the updated value appears correctly in Notion (not the original value).

---

## Gap 4 — Phase 9's "queues locally and retries" on Notion outage was never actually built

**Problem:** The Phase 9 test list includes an edge case: *"Notion API down/rate-limited → bot queues the write locally and retries, tells user 'saved locally, syncing shortly'."* But Phase 3's SQLite schema was only ever designed as a **read mirror**, not a write-outbox — there's no table or worker that actually holds an unsent write and retries it.

**This matters because:** if the bot currently tells you "saved" when a Notion call actually failed, that's a false claim about your real data — worse than an honest error message.

**Audit first:** check what currently happens if `notion_client_wrapper.create_page()` raises `NotionAPIError` / `NotionRateLimitError` on Confirm. If it currently just fails and tells the user honestly, that's fine for now — skip to the fix's "minimum" option.

**Fix — minimum viable (do this now):** on Notion write failure, tell the user plainly it failed and ask them to retry ("⚠️ Notion didn't respond, nothing was saved — tap Confirm again in a moment") and **do not** claim it's saved anywhere. Keep the draft in `pending_writes` (Gap 2) so Confirm can just be tapped again without re-typing.

**Fix — full version (defer unless you want it now):** add a real `write_outbox` table + a background retry worker with backoff. Only worth doing if Notion outages are actually a recurring problem in practice — don't build it speculatively.

**DoD (minimum version):** simulate a Notion failure (bad token temporarily), confirm the bot's message is honest about the failure and the draft survives for a retry.

---

## Gap 5 — Clarification questions don't know what they're answers to

**Problem:** (from earlier discussion) When `needs_clarification: true` triggers a question, the next message you send gets parsed cold — the intent parser has no idea it's a reply to the question it just asked.

**Fix:** add one field to the existing per-chat session context (same table Phase 6 already built for subject/chapter/block): `pending_clarification: {original_message, question, asked_at}`. When a new message arrives and this field is set, pass **both** the original message and the question, plus the new reply, into the next `intent_parser` call so it can resolve the combined intent. Clear the field once resolved (or re-ask if the answer is still ambiguous).

**DoD:** trigger a clarification (ambiguous chapter name), reply with a short answer like "the friction one," confirm the resulting draft correctly reflects the *original* logging intent plus the clarified chapter — not a fresh, unrelated parse of just the reply.

---

## Gap 6 — Fuzzy-match ambiguity on relations has no defined resolution path

**Problem:** Phase 7 step 5 says relation fields get fuzzy-matched against the mirror to resolve name → page\_id, but doesn't say what happens when the match is ambiguous (e.g. two similarly-named chapters).

**Fix:** define a threshold — if the top match's confidence isn't clearly ahead of the second-best candidate, treat

it as ambiguous and route into the **same clarification mechanism as Gap 5**, but present the candidates as inline buttons (not a free-text question) so the user just taps the correct one instead of retyping.

**DoD:** create a test scenario with two similarly-named chapters in the mirror, confirm the bot asks you to pick rather than silently guessing (or silently picking the wrong one).

---

## Gap 7 (lower priority, infra hygiene) — Sync and write-triggered sync could race

**Problem:** The scheduled sync loop (every 3–5 min) and the on-write incremental sync (Phase 3 step 3) both touch the same SQLite file. No locking was specified.

**Fix:** enable WAL mode on the mirror DB ( `PRAGMA journal_mode=WAL;` ) to reduce reader/writer contention, and wrap sync operations in a single lock ( `asyncio.Lock` ) so the scheduled loop and the write-triggered sync never run concurrently.

**DoD:** trigger a manual log entry at the same moment the scheduled sync would normally fire (or just run



both back-to-back rapidly a few times); confirm no  
`database is locked` errors in logs.

---

## Closing note for kimi-code

None of these are new features — they're gaps in the original spec at the seams *between* already-built phases. Audit each one against the actual current code first, fix only what's genuinely missing, run each gap's DoD, then Phase 10 (hardening/deploy) can close for real.